

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 1)

MAHALINGAM Nithees (exercise 2)

SARAVANAN Arun Prasath (exercise 3)

Github Link:

https://github.com/anshuman-krishna/advanced-algorithms-and-programming/tree/LAB_06_BasicsOfGraphs

Exercise 1: YADAV Anshuman Krishna

Graph Representations for Social Networks

1. SocialGraph Structure

STRUCTURE SocialGraph

matrix <- empty 2D array

adj_list <- empty dictionary

num_users <- 0

num_edges <- 0

END STRUCTURE

Time Complexity : $O(1)$

Space Complexity : $O(1)$

2. Adjacency Matrix Operations

A. add_friendship_matrix

PROCEDURE add_friendship_matrix(graph, u, v)

graph.matrix[u][v] <- 1

graph.matrix[v][u] <- 1

graph.num_edges <- graph.num_edges + 1

END PROCEDURE

Time Complexity : $O(1)$

Space Complexity : $O(1)$, auxiliary space.

B. remove_friendship_matrix

PROCEDURE remove_friendship_matrix(graph, u, v)

graph.matrix[u][v] <- 0

graph.matrix[v][u] <- 0

graph.num_edges <- graph.num_edges - 1

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

END PROCEDURE

Time Complexity : $O(1)$, direct array assignment

Space Complexity : $O(1)$

C. are_friends_matrix

FUNCTION are_friends_matrix(graph, u, v) -> Boolean

IF graph.matrix[u][v] = 1 THEN

RETURN TRUE

END IF RETURN FALSE

END FUNCTION

Time Complexity : $O(1)$, direct lookup

Space Complexity : $O(1)$

D. get_friends_matrix

FUNCTION get_friends_matrix(graph, u) -> List

friends <- empty list

FOR i <- 0 TO graph.num_users - 1 DO

IF graph.matrix[u][i] = 1 THEN

friends.Add(i)

END IF END FOR

RETURN friends

END FUNCTION

Time Complexity : $O(V)$, V is the total number of users. Need to scan the entire row.

Space Complexity : $O(V)$, to store the returned list of friends.

E. get_degree_matrix

FUNCTION get_degree_matrix(graph, u) -> Integer

count <- 0

FOR i <- 0 TO graph.num_users - 1 DO

IF graph.matrix[u][i] = 1 THEN

count <- count + 1

END IF END FOR

RETURN count

END FUNCTION

Time Complexity : $O(V)$, need to iterate through all the users.

Space Complexity : $O(1)$

3. Adjacency List Operations

A. add_friendship_list

```
PROCEDURE add_friendship_list(graph, u, v)
graph.adj_list[u].Add(v)
graph.adj_list[v].Add(u)
graph.num_edges <- graph.num_edges + 1
END PROCEDURE
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$, auxiliary.

B. remove_friendship_list

```
PROCEDURE remove_friendship_list(graph, u, v)
graph.adj_list[u].Remove(v)
graph.adj_list[v].Remove(u)
graph.num_edges <- graph.num_edges - 1
END PROCEDURE
```

Time Complexity : $O(D)$, D is degree (no. of friends). Need to search through u 's list to find v before removing it.

Space Complexity : $O(1)$

C. are_friends_list

```
FUNCTION are_friends_list(graph, u, v) -> Boolean
IF graph.adj_list[u] contains v THEN
RETURN TRUE
END IF RETURN FALSE
END FUNCTION
```

Time Complexity : $O(D)$, D is degree. Need to linearly scan the friends list.

Space Complexity : $O(1)$

D. get_friends_list

```
FUNCTION get_friends_list(graph, u) -> List
RETURN graph.adj_list[u]
END FUNCTION
```

Time Complexity : $O(1)$, the list already exists in the dictionary, need to return it.

Space Complexity : $O(D)$, for the returned array.

E. get_degree_list

FUNCTION get_degree_list(graph, u) -> Integer

RETURN length(graph.adj_list[u])

END FUNCTION

Time Complexity : $O(1)$

Space Complexity : $O(1)$

4. Graph Property and Conversions

A. get_num_users

Time Complexity :

Space Complexity :

B. get_num_edges

Time Complexity :

Space Complexity :

C. is_complete_graph

Time Complexity :

Space Complexity :

D. graph_density

Time Complexity :

Space Complexity :

E. degree_distribution

Time Complexity :

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

Space Complexity :

```
F. matrix_to_list
PROCEDURE matrix_to_list(graph)
graph.adj_list <- empty dictionary
FOR i <- 0 TO graph.num_users - 1 DO
graph.adj_list[i] <- empty list
FOR j <- 0 TO graph.num_users - 1 DO
IF graph.matrix[i][j] = 1 THEN
graph.adj_list[i].Add(j)
END IF
END FOR
END FOR
END PROCEDURE
```

Time Complexity : $O(V^2)$, forced to check every single cell in 2D matrix grid.

Space Complexity : $O(V + E)$, to build new list in memory.

```
G. list_to_matrix
PROCEDURE list_to_matrix(graph)
graph.matrix <- empty 2D array
FOR i <- 0 TO graph.num_users - 1 DO
FOR j <- 0 TO graph.num_users - 1 DO
graph.matrix[i][j] <- 0
END FOR
END FOR
FOR EACH u IN graph.adj_list DO
FOR EACH v IN graph.adj_list[u] DO
graph.matrix[u][v] <- 1
END FOR
END FOR
END PROCEDURE
```

Time Complexity : $O(V^2)$, to initialize the empty grid and populate it.

Space Complexity : $O(V^2)$, new matrix.

Complexity Analysis Questions:

1. What is the time complexity for `are_friends(u, v)` in each representation? Prove it.

Answer:

Adjacency Matrix: $O(1)$.

Since the matrix is a 2D array, we just plug in the indexes `matrix[u][v]` and check if it equals 1. Memory lookup is instantaneous.

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

Adjacency List: $O(V)$.

To find out if v is friends with u , we have to go to u 's list and search through it one-by-one until we find v . In the very absolute worst case (if u is friends with almost everyone), we scan roughly V items.

2. For a social network with 1 billion users where the average user has 150 friends, which representation is more space efficient? Calculate the exact memory difference.

Answer:

- Matrix: 1 billion rows by 1 billion columns equals 10^{18} cells. If each cell takes 1 byte, that is 10^{18} bytes of memory.
- List: 1 billion users + (1 billion users x 150 friends) = 150 billion connections. If we assume an ID is about 1 byte, that's roughly 150×10^9 bytes.

Conclusion: The Adjacency List is vastly more efficient.

3. If you need to frequently add/remove friendships, which representation is better? Why?

Answer:

Well, on paper, the Adjacency Matrix is technically better for adding and removing because it does both in $O(1)$ time (just flipping a 1 to a 0). The Adjacency List takes $O(V)$ to remove an edge because you have to search the list to find the ID to delete.

However, for an app like Instagram (our project), the Matrix is impossible to use because of the memory constraints we just discussed above. So, practically, we must use the Adjacency List.

Also, a good compromise we learned is using an Adjacency List but replacing the arrays with Hash Sets, which gives us great memory efficiency.

Exercise 2: MAHALINGAM Nithees

Graph Traversals - DFS for Social Network Analysis

1. DFS Traversals

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

Part A: Recursive DFS

```
dfs_recursive(graph, start):
    visited = empty set
    result = empty list
    function dfs(node):
        if node not in visited:
            add node to visited
            add node to result
            for each neighbor in graph[node]:
                dfs(neighbor)
    dfs(start)
    return result
```

Part B: Iterative DFS using stack

```
dfs_iterative(graph, start):
    visited = empty set
    stack = [start]
    result = empty list
    while stack is not empty:
        node = stack.pop()
        if node not in visited:
            add node to visited
            add node to result
            for each neighbor in reverse(graph[node]):
                stack.push(neighbor)
    return result
```

Part C: Find all connected components

```
find_connected_components(graph):
    visited = empty set
    components = empty list
    for each node in graph:
        if node not in visited:
            component = dfs_recursive(graph, node)
            add all nodes of component to visited
            add component to components
    return components
```

Part D: Check if graph is connected

```
is_connected(graph):
    start = any node from graph
```

```
visited_nodes = dfs_recursive(graph, start)
if size of visited_nodes == total nodes:
    return true
else:
    return false
```

Part E: Find if path exists between two users

```
has_path(graph, start, target):
    visited = empty set
    function dfs(node):
        if node == target:
            return true
        add node to visited
        for each neighbor in graph[node]:
            if neighbor not in visited:
                if dfs(neighbor) == True:
                    return True
        return false
    return dfs(start)
```

Part F: Find path between users

```
find_path(graph, start, target):
    visited = empty set
    parent = empty map
    function dfs(node):
        add node to visited
        if node == target:
            return true
        for each neighbor in graph[node]:
            if neighbor not in visited:
                parent[neighbor] = node
                if dfs(neighbor):
                    return true
        return false
    dfs(start)
    if target not in parent and start != target:
        return empty list

    path = []
    current = target
    while current exists:
        add current to path
```

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

```
current = parent[current]
reverse(path)
return path
```

2. Travel Based Analytics:

.../bfm

Complexity Analysis Questions:

1. What is the time and space complexity of DFS on an adjacency list representation? On adjacency matrix?

Answer:

	Adjacency List	Adjacency Matrix
Time Complexity	$O(V + E)$ visiting each node + each edge once	$O(V^2)$ checks all possible edges
Space Complexity	$O(V)$ visited + recursion/stack	$O(V^2)$ matrix storage

2. Why might recursive DFS cause problems in a social network with 1 million users? How would you solve it?

Answer:

Recursive DFS can cause problems in a social network with 1 million users because it uses the call stack, which has limited memory. If the graph is very deep, it may lead to a stack overflow error and crash the program.

Solution:

Use iterative DFS with a stack or BFS (queue) instead of recursion. These approaches use heap memory and can handle large graphs without recursion limits.

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

Exercise 3: SARA VANAN Arun Prasath

Graph Traversals - BFS for Shortest Path Analysis

Graph Class Setup

```
from collections import deque, defaultdict
class SocialGraph:
    def __init__(self):
        self.graph = defaultdict(list)
    def add_user(self, user):
        if user not in self.graph:
            self.graph[user] = []
    def add_friendship(self, user1, user2):
        self.graph[user1].append(user2)
        self.graph[user2].append(user1)
```

Part A - Basic BFS

```
def bfs(self, start_user):
    visited = set()
    queue = deque([start_user])
    order = []
    visited.add(start_user)
    while queue:
        user = queue.popleft()
        order.append(user)
        for friend in self.graph[user]:
            if friend not in visited:
                visited.add(friend)
                queue.append(friend)
    return order
```

Part B — BFS with Distances

```
def bfs_with_distances(self, start_user):
    visited = set([start_user])
    queue = deque([start_user])
    distance = {start_user: 0}
    while queue:
        user = queue.popleft()
        for friend in self.graph[user]:
            if friend not in visited:
                visited.add(friend)
                queue.append(friend)
```

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

```
distance[friend] = distance[user] + 1
return distance
```

Part C - Shortest Path

```
def shortest_path(self, start_user, target_user):
    visited = set([start_user])
    queue = deque([start_user])
    parent = {start_user: None}
    while queue:
        user = queue.popleft()
        if user == target_user:
            break
    for friend in self.graph[user]:
        if friend not in visited:
            visited.add(friend)
            queue.append(friend)
            parent[friend] = user
    if target_user not in parent:
        return []
    path = []
    current = target_user
    while current is not None:
        path.append(current)
        current = parent[current]
    return path[::-1]
```

Part D — Degrees of Separation

```
def degrees_of_separation(self, start_user, target_user):
    distances = self.bfs_with_distances(start_user)
    return distances.get(target_user, -1)
```

Part E — Users Within K Hops

```
def friends_within_k_hops(self, start_user, k):
    distances = self.bfs_with_distances(start_user)
    return {user for user, d in distances.items() if d <= k}
```

2.Implement traversal-based analytics:

Average Degrees:

```
def compute_average_degrees_of_separation(self):
    total = 0
    count = 0
```

```
for user in self.graph:
    distances = self.bfs_with_distances(user)
    for d in distances.values():
        total += d
    count += 1
return total / count if count else 0
```

Distance Distribution:

```
def get_distance_distribution(self, start_user):
    distances = self.bfs_with_distances(start_user)
    distribution = {}
    for d in distances.values():
        distribution[d] = distribution.get(d, 0) + 1
    return distribution
```

Friend Recommendation:

```
def recommend_friends(self, start_user, max_recommendations=5):
    distances = self.bfs_with_distances(start_user)
    suggestions = [user for user, d in distances.items() if d == 2]
    return suggestions[:max_recommendations]
```

Complexity Analysis Questions:

1. What is the time and space complexity of BFS? How does it compare to DFS?

Answer:

The time complexity of **BFS is $O(V + E)$** , where V is the number of vertices and E is the number of edges. This is because BFS visits each node once and checks all its connections.

The space complexity is **$O(V)$** , since it stores visited nodes, a queue, and sometimes distance/parent data.

Comparison with DFS:

DFS also has $O(V + E)$ time complexity. However, BFS uses a queue and can take more memory in wide graphs, while DFS uses a stack/recursion and is often more memory-efficient.

2. In an unweighted social network, why is BFS guaranteed to find the shortest

II.2415 Advanced Algorithms and Programming

Lab 6: Basics of Graph Structures: Team T24

Path?

Answer:

In an **unweighted graph**, all edges have equal cost (1 step). BFS explores nodes **level by level**, starting from the source.

It first visits nodes at **distance 1**, then **distance 2**, and so on. Because of this, the **first time a node is reached** is always through the **shortest path**.

3. For finding "six degrees of separation" between two users, would you use BFS or DFS? Why?

Answer:

.../bfm

4. In a social network, would you use DFS or BFS to find the shortest friendship chain? Why?

Answer:

.../bfm

Final Integration Question

to be continued in T24_LAB6_BFM